

NAMA : Fajar Firdaus  
NPM : 242310006  
KELAS : TI-24-PA1  
MATKUL : Lab. Pemrograman Web  
LINK GITHUB : <https://github.com/jars-ops/PW-MERN-STACK-242310006>

## Membuat file userController

```
1  const db = require("../models");
2  const User = db.User;
3
4  exports.getAllUsers = async (req, res) => {
5    try {
6      const users = await User.findAll();
7      res.json({ success: true, data: users });
8    } catch (error) {
9      res.status(500).json({ success: false, message: error.message });
10   }
11 };
12
13 exports.getUserById = async (req, res) => {
14   try {
15     const user = await User.findByPk(req.params.id);
16     if (!user) return res.status(404).json({ success: false, message: "User not found" });
17     res.json({ success: true, data: user });
18   } catch (error) {
19     res.status(500).json({ success: false, message: error.message });
20   }
21 };
22
23 exports.createUser = async (req, res) => {
24   try {
25     const newUser = await User.create(req.body);
26     res.status(201).json({ success: true, message: "User created", data: newUser });
27   } catch (error) {
28     res.status(500).json({ success: false, message: error.message });
29   }
30 };
31
32 exports.updateUser = async (req, res) => {
33   try {
34     const user = await User.findByPk(req.params.id);
35     if (!user) return res.status(404).json({ success: false, message: "User not found" });
36
37     await user.update(req.body);
38     res.json({ success: true, message: "User updated", data: user });
39   } catch (error) {
40     res.status(500).json({ success: false, message: error.message });
41   }
42 };
43
44 exports.deleteUser = async (req, res) => {
45   try {
46     const user = await User.findByPk(req.params.id);
47     if (!user) return res.status(404).json({ success: false, message: "User not found" });
48
49     await user.destroy();
50     res.json({ success: true, message: "User deleted" });
51   } catch (error) {
52     res.status(500).json({ success: false, message: error.message });
53   }
54 };
```

## Membuat file userRoutes

```
1  const express = require("express");
2  const router = express.Router();
3  const userController = require("../controllers/userController");
4
5  router.get("/", userController.getAllUsers);
6  router.get("/:id", userController.getUserById);
7  router.post("/", userController.createUser);
8  router.put("/:id", userController.updateUser);
9  router.delete("/:id", userController.deleteUser);
10
11 module.exports = router;
```

## Menambahkan userRoutes pada index.js

```
backend > index.js > ...
1 | require("dotenv").config();
2 | const express = require("express");
3 | const cors = require("cors");
4 | const db = require("../models");
5 | const app = express();
6 | // Middleware
7 | app.use(cors({
8 |   origin: "http://localhost:3000",
9 |   credentials: true
10 | }));
11 | app.use(express.json());
12 | app.use(express.urlencoded({ extended: true }));
13 |
14 | const bookRoutes = require("../routes/bookRoutes");
15 | app.use("/api/books", bookRoutes);
16 |
17 | const userRoutes = require("../routes/userRoutes");
18 | app.use("/api/users", userRoutes);
19 |
20 | // Test database connection
21 | db.sequelize.authenticate()
22 |   .then(() => {
23 |     console.log("✓ Koneksi ke database MySQL berhasil!");
```

## Menambah User Baru (POST)

The screenshot shows a Postman interface with a new request configured as a POST to `http://localhost:3001/api/users`. The body is in raw JSON format, containing user details. The response is a 201 status code, indicating successful creation.

**Request Body:**

```
{
  "email": "fajar@mail.com",
  "username": "sanji14",
  "password": "jst123",
  "is_active": true
}
```

**Response Body:**

```
{
  "success": true,
  "message": "User created",
  "data": {
    "id": 1,
    "email": "fajar@mail.com",
    "username": "sanji14",
    "password": "jst123",
    "is_active": true,
    "updatedAt": "2024-07-14T14:57:51.522Z",
    "createdAt": "2024-07-14T14:57:51.522Z"
  }
}
```

## Mengambil Semua Daftar User (GET)

The screenshot shows a Postman interface with a new request configured as a GET to `http://localhost:3001/api/users`. The response is a 200 status code, indicating successful retrieval of all users.

**Response Body:**

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "email": "fajar@mail.com",
      "username": "sanji14",
      "password": "jst123",
      "is_active": true,
      "updatedAt": "2024-07-14T14:57:51.088Z",
      "createdAt": "2024-07-14T14:57:51.088Z"
    }
  ]
}
```

## Mengambil Detail User Berdasarkan ID

The screenshot shows a Postman interface with a new collection named "Mengambil Detail User Berdasarkan ID". A GET request is configured to `http://localhost:3001/api/users/1`. The response is a 200 OK status with a JSON body:

```
{
  "success": true,
  "data": {
    "id": 1,
    "email": "fajar@mail.com",
    "username": "sanj154",
    "password": "js123",
    "is_active": true,
    "createdAt": "2020-07-16T14:57:51.000Z",
    "updatedAt": "2020-07-16T14:57:51.000Z"
  }
}
```

## Memperbarui Data User (PUT)

The screenshot shows a Postman interface with a new collection named "Memperbarui Data User". A PUT request is configured to `http://localhost:3001/api/users/1`. The request body is a JSON object:

```
{
  "username": "shanks"
}
```

The response is a 200 OK status with a JSON body:

```
{
  "success": true,
  "message": "User updated",
  "data": {
    "id": 1,
    "email": "fajar@mail.com",
    "username": "shanks",
    "password": "js123",
    "is_active": true,
    "createdAt": "2020-07-16T14:57:51.000Z",
    "updatedAt": "2020-07-16T15:09:29.000Z"
  }
}
```

## Menghapus User (DELETE)

The screenshot shows a Postman interface with a new collection named "Menghapus User". A DELETE request is configured to `http://localhost:3001/api/users/1`. The response is a 200 OK status with a JSON body:

```
{
  "success": true,
  "message": "User deleted"
}
```